

# Lecture 24: Convolutional neural networks part 2

**Liberty Hamilton**

**April 23, 2026**

*h/t to NeuroMatch academy (Alona Fyshe)*

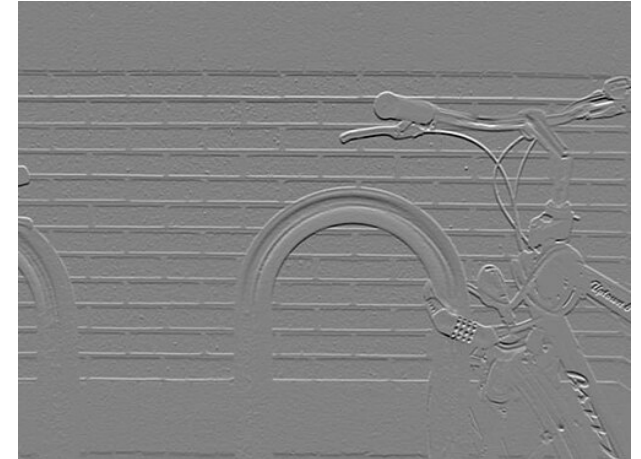
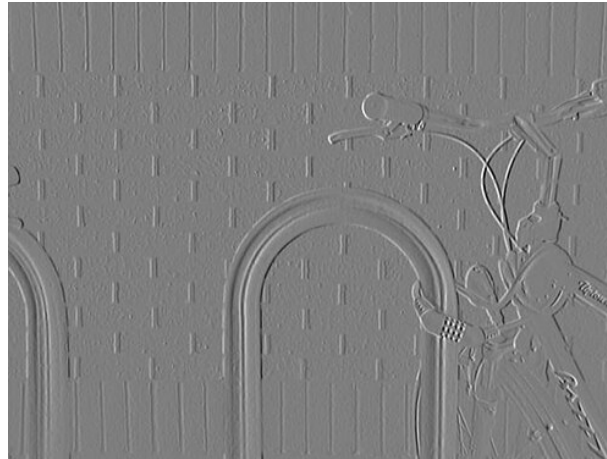
# What do you think would happen?

Given an image  $A$  and the kernel  $K$  below

Image  $A$



$$\mathbf{G}_x = \overset{K}{\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}} * \mathbf{A} \quad \mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \mathbf{A}$$



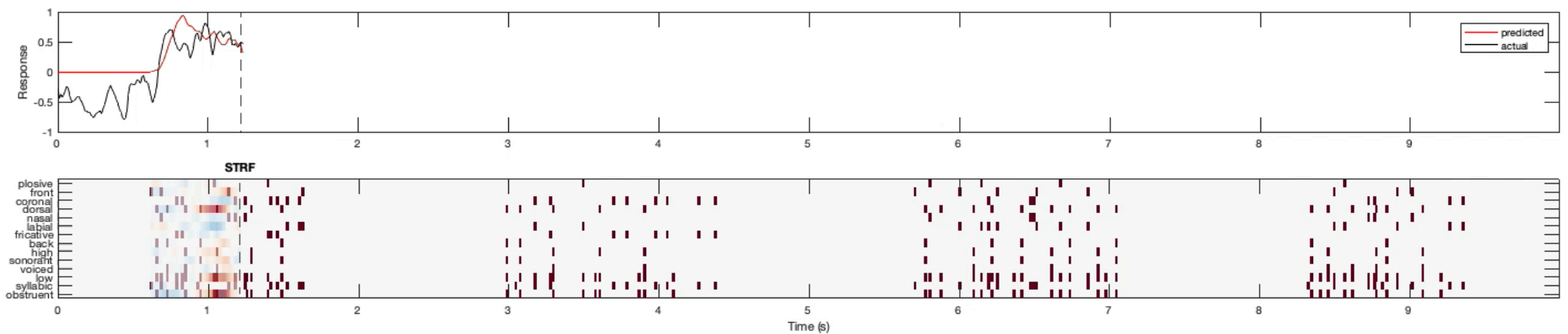
# But how do we get all the edges?

- We need a nonlinearity!
- In CNNs, this is typically done using a ReLU (rectified linear unit)
  - $\text{ReLU}(x) = \max(0, x)$

nonlinear combination of  $G_x$  and  $G_y$



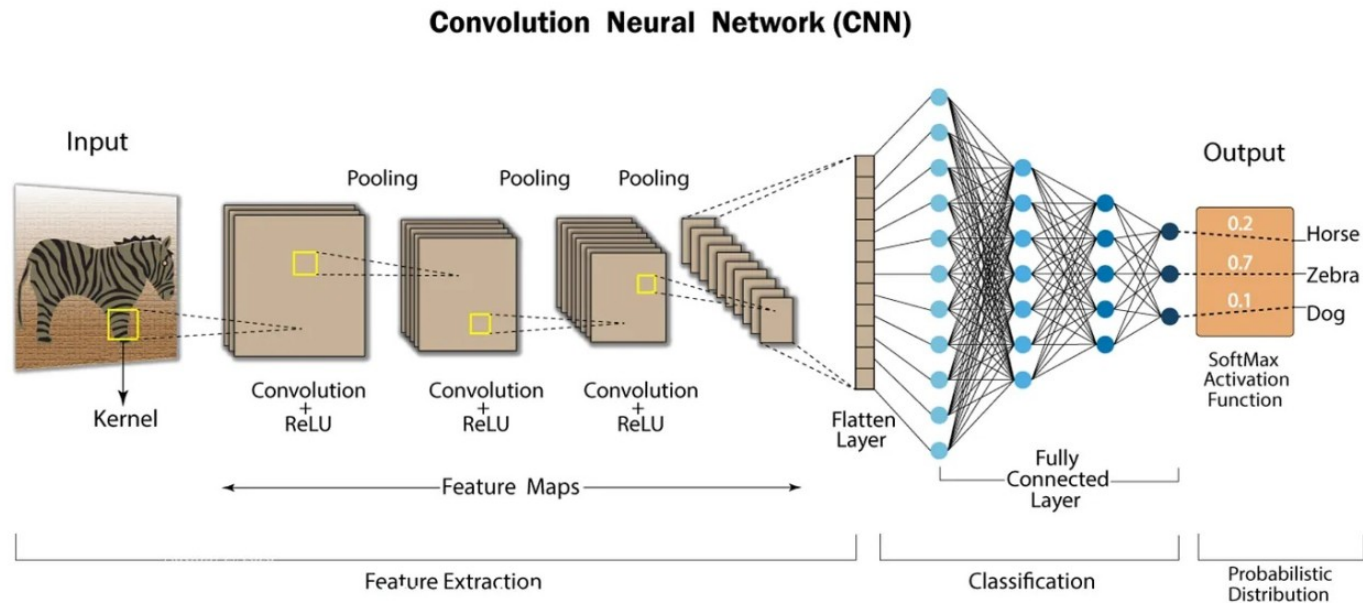
# STRF and time lagged regression as convolution



- Here we are convolving our 2D STRF filter (the beta weights from time lagged regression) with our feature matrix to produce the predicted neural data (red trace on top)

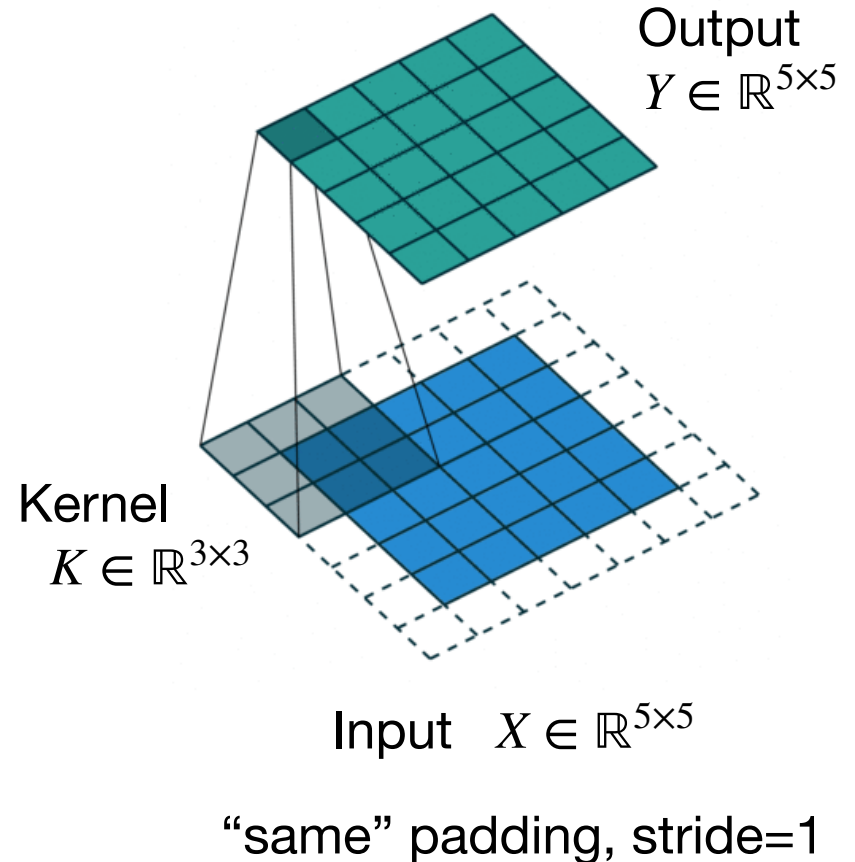
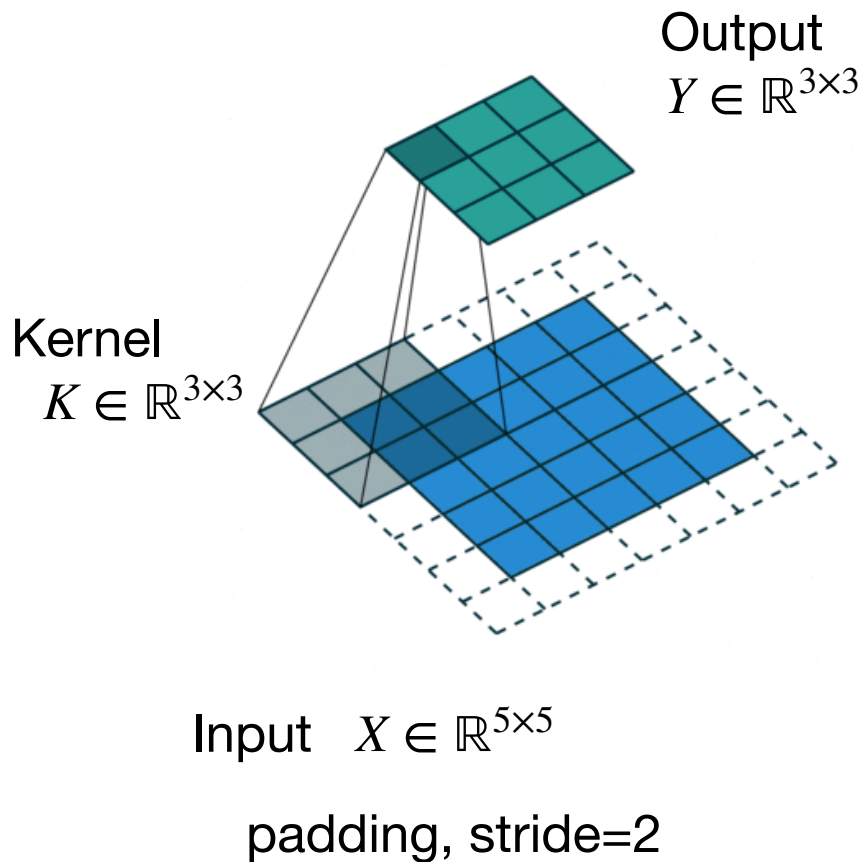
**What are the building blocks of CNNs?**

# Elements of a CNN



1. Input layer
2. Convolutional layer
3. Activation layer (introduces nonlinearity)
4. Pooling layer
5. Flattening
6. Fully connected (dense) layer
7. Output layer

# Example convolution operations (from last time)



## Combining multiple filters

**Input X (9x9)**

[illegible]

**Kernel K (3x3)**

|    |    |    |
|----|----|----|
| +1 | -1 | -1 |
| -1 | +1 | -1 |
| -1 | -1 | +1 |

### Output Y (7x7)

A 10x10 grid with a red square in the top-left corner containing the text '+9'.

size =  $(9-3+1) \times (9-3+1)$

**Input X (9x9)**

[illegible]

**Kernel K (3x3)**

|    |    |    |
|----|----|----|
| -1 | -1 | +1 |
| -1 | +1 | -1 |
| +1 | -1 | -1 |

### Output Y (7x7)

## Combining multiple filters

**Input X (9x9)**

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| +1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | +1 |
| -1 | +1 | -1 | -1 | -1 | -1 | -1 | +1 | -1 |
| -1 | -1 | +1 | -1 | -1 | -1 | +1 | -1 | -1 |
| -1 | -1 | -1 | +1 | -1 | +1 | -1 | -1 | -1 |
| -1 | -1 | -1 | -1 | +1 | -1 | -1 | -1 | -1 |
| -1 | -1 | -1 | +1 | -1 | +1 | -1 | -1 | -1 |
| -1 | -1 | +1 | -1 | -1 | -1 | +1 | -1 | -1 |
| -1 | +1 | -1 | -1 | -1 | -1 | -1 | +1 | -1 |
| +1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | +1 |

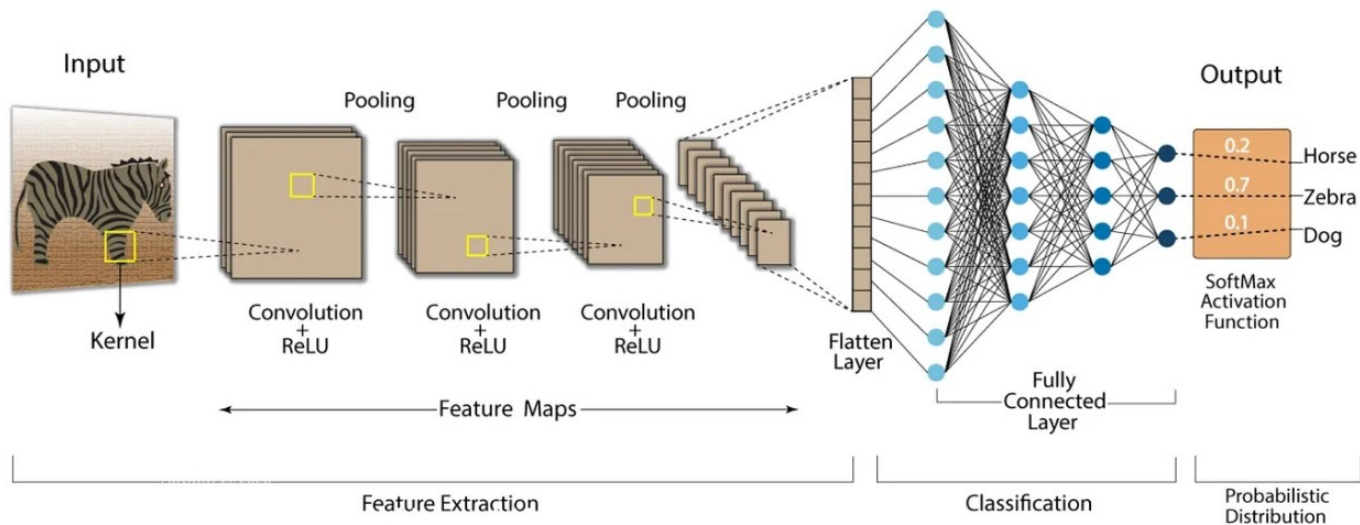
### Kernel K (3x3)

|    |    |    |
|----|----|----|
| +1 | -1 | +1 |
| -1 | +1 | -1 |
| +1 | -1 | +1 |

### Output Y (7x7)

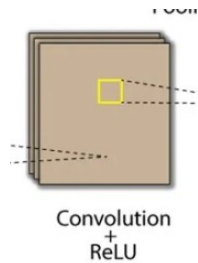
# Adding a nonlinearity

Convolution Neural Network (CNN)

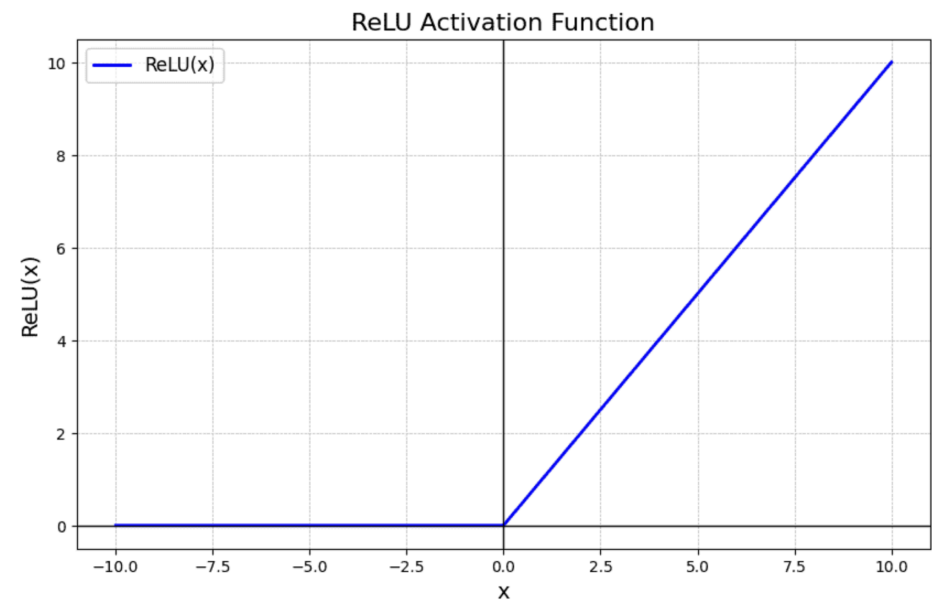


1. Input layer
2. Convolutional layer
3. **Activation layer (introduces nonlinearity)**
4. Pooling layer
5. Flattening
6. Fully connected (dense) layer
7. Output layer

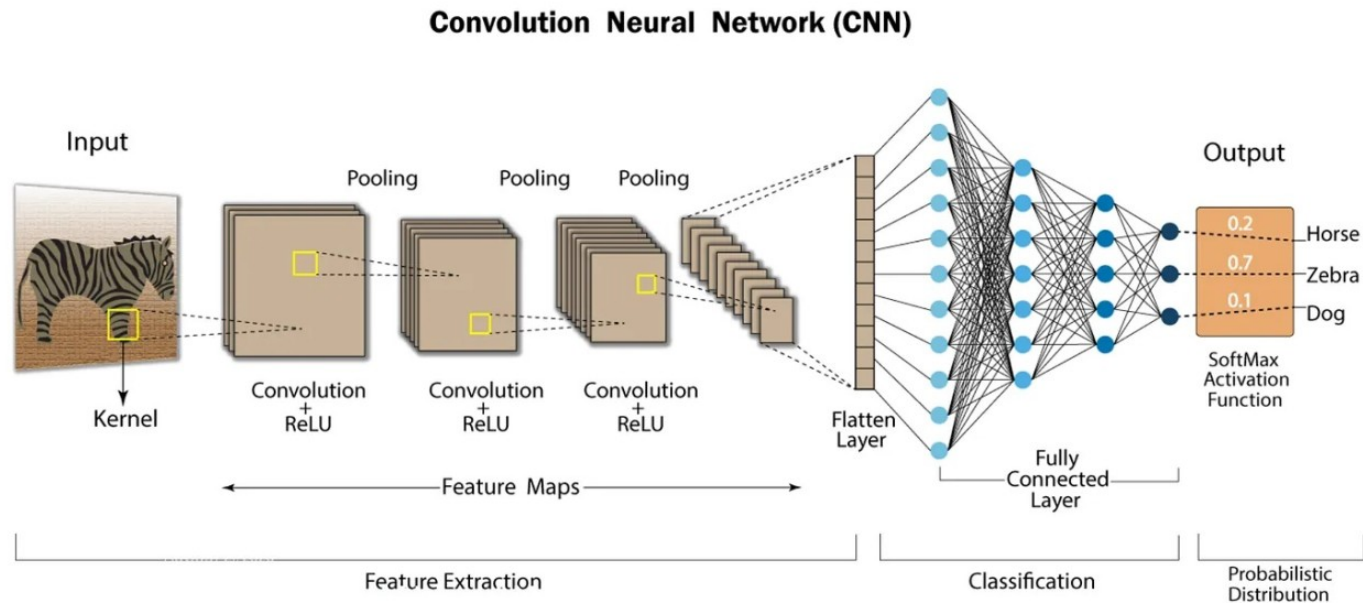
# Adding a nonlinearity



- ReLU = rectified linear unit
- What does this do?
  - $\text{ReLU}(x) = \max(0, x)$
- Introduces sparseness
- Helps manage vanishing gradient problem
- Allows for compositionality



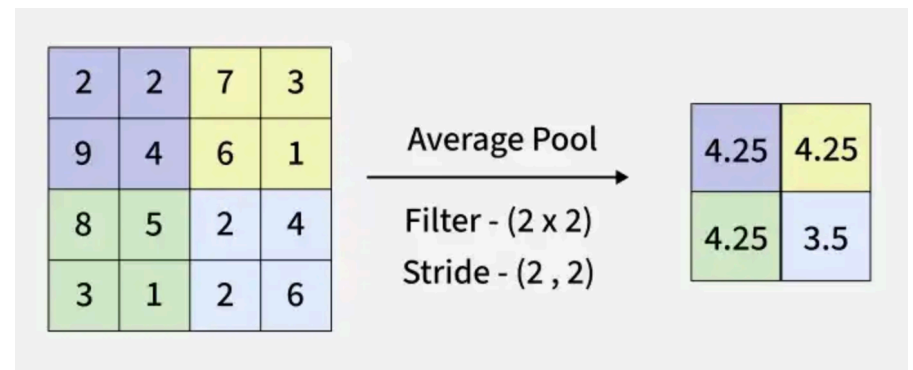
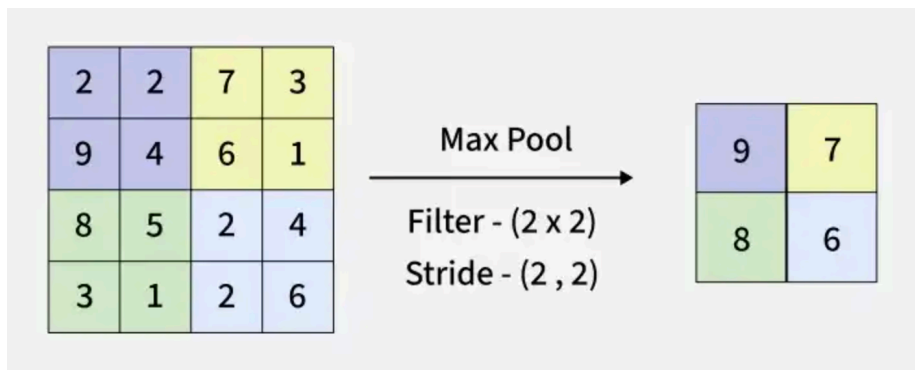
# Elements of a CNN



1. Input layer
2. Convolutional layer
3. Activation layer (introduces nonlinearity)
4. **Pooling layer**
5. Flattening
6. Fully connected (dense) layer
7. Output layer

# Pooling layer

- We can introduce pooling to reduce the dimensionality of feature maps
- Common: max-pooling or average-pooling
- Helps with translation invariance (small shifts in image don't affect output)



# Image classification

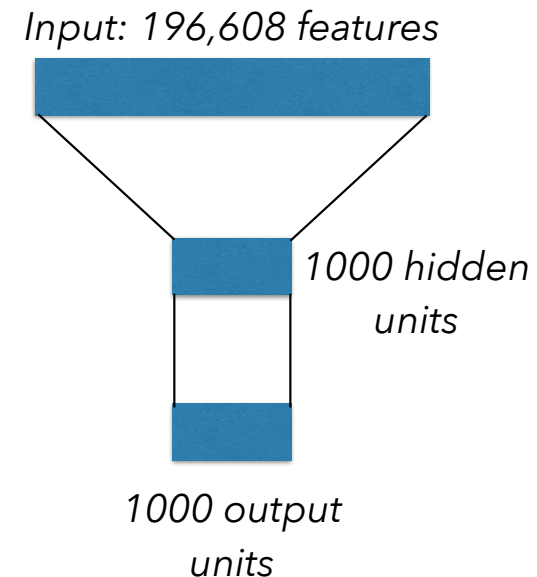
- Formalization of the image classification problem:
  - **Input:** an image (256 x 256 RGB pixels = 196,608 features)
  - **Target:** a 1-hot vector (e.g.  $[0, 1, 0, 0, 0, \dots, 0]$ ) indicating the “class” or “label” of the image



↓  
 $[1, 0, \dots, 0]$

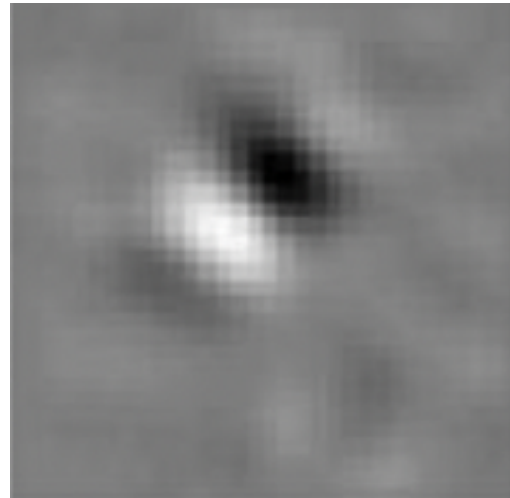
# Image classification

- How should a neural network be designed to solve this problem?
- Suppose we used a simple network with 1 hidden layer containing 1000 units, and 1000 image classes
- How many weights would this network contain?



# Image Classification

- **Idea 1:** localized receptive fields
- We know from neuroscience that visual neurons have limited *receptive fields*



# Image Classification

- So: instead of having the entire image feed into each hidden unit, have each unit only look at **a small portion** of the image, e.g. 11x11 pixels (363 weights)
- We can call this the **receptive field** for the unit
- Let's offset each 11x11 unit by 4 pixels, giving 4096 “localized” units



# Image Classification

- So: instead of having the entire image feed into each hidden unit, have each unit only look at **a small portion** of the image, e.g. 11x11 pixels (363 weights)
- We can call this the **receptive field** for the unit
- Let's offset each 11x11 unit by 4 pixels, giving 4096 “localized” units



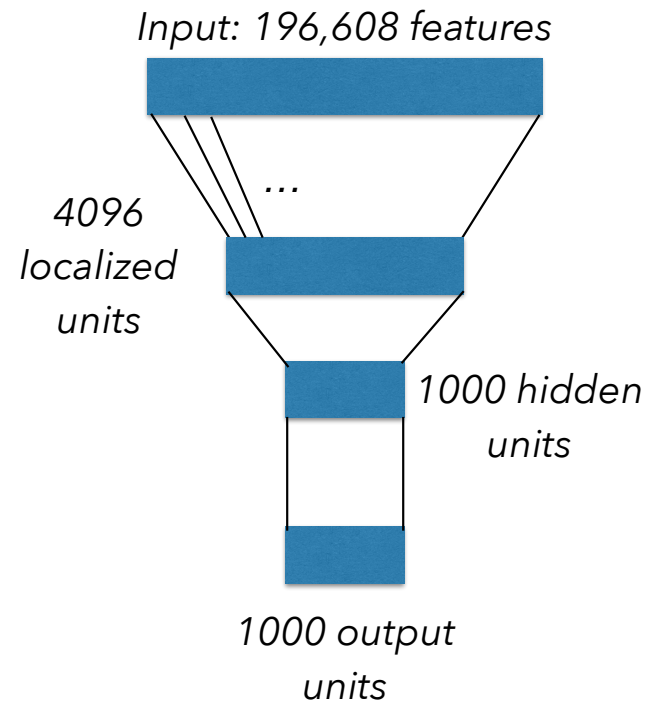
# Image Classification

- So: instead of having the entire image feed into each hidden unit, have each unit only look at **a small portion** of the image, e.g. 11x11 pixels (363 weights)
- We can call this the **receptive field** for the unit
- Let's offset each 11x11 unit by 4 pixels, giving 4096 “localized” units



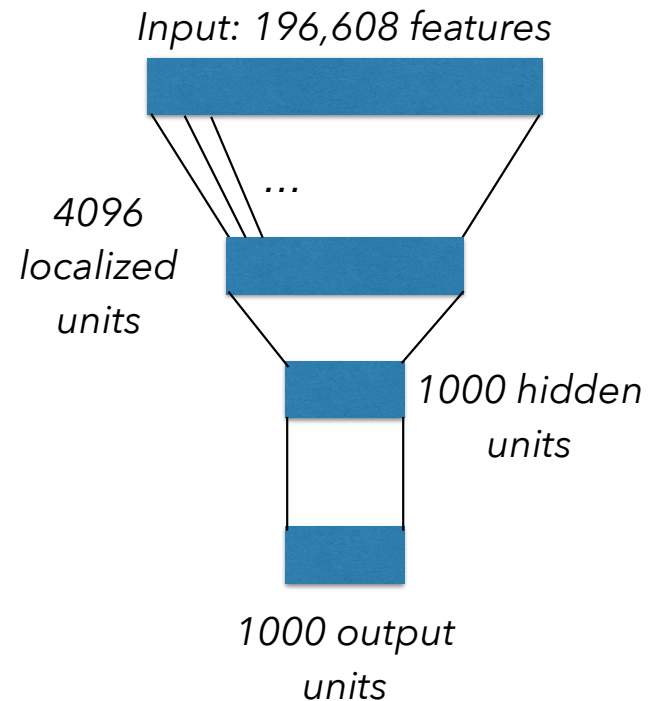
# Image Classification

- So: instead of having the entire image feed into each hidden unit, have each unit only look at **a small portion** of the image, e.g. 11x11 pixels (363 weights)
- Let's offset each 11x11 unit by 4 pixels, giving 4096 “localized” units
- To combine info. across these patches let's keep our original 1000-unit layer also
- How many total weights now?



# Image Classification

- This localized network is pretty small, but still not very **expressive** (it only detects one “feature” at each location!)
- Stacking more “localized” or “fully-connected” layers (or making them wider) could improve things, but at substantial cost



# Translation invariance



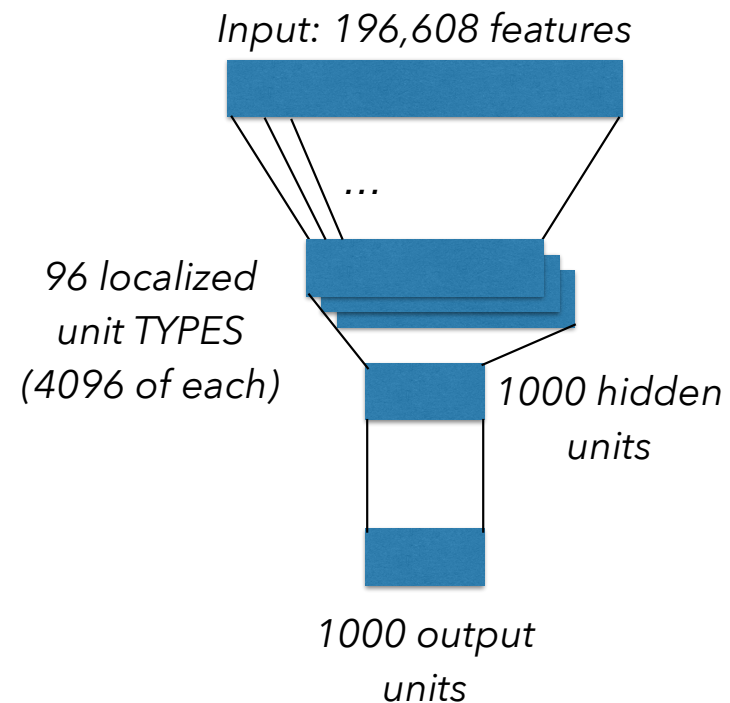
- Object classes are *translation invariant*
- i.e., object class (mostly) doesn't depend on where the object is within the image

# Convolutional layer

- Idea 2: *translation invariant networks*
- Instead of learning a separate set of weights for each “localized” unit, let’s learn **one** set of weights that is **shared** across all the localized units
- This operation—applying the same “filter” to each patch in the image—resembles **convolution** of the image with a filter/kernel
  - So let’s call it a “convolutional layer”

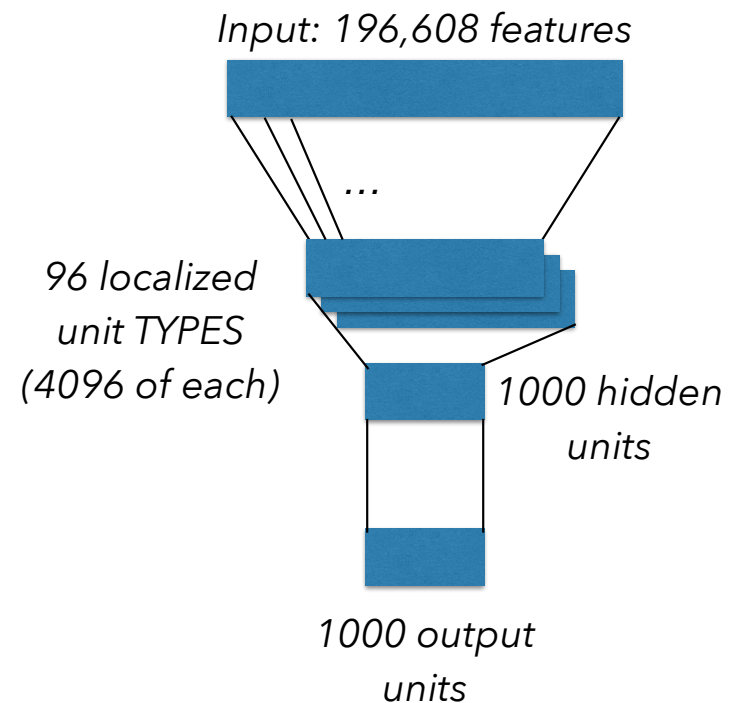
# Convolutional layer

- Using convolution makes it easy to add more “feature detectors” for each location in the image
- Instead of training 4096 detectors (one for each location), we can just train (e.g.) 96, each of which is **shared** across the 4096 locations



# Convolutional layer

- This network is better, and doesn't have too many weights!
- But it's still not very expressive
- This could be improved by making it deeper, but that's expensive



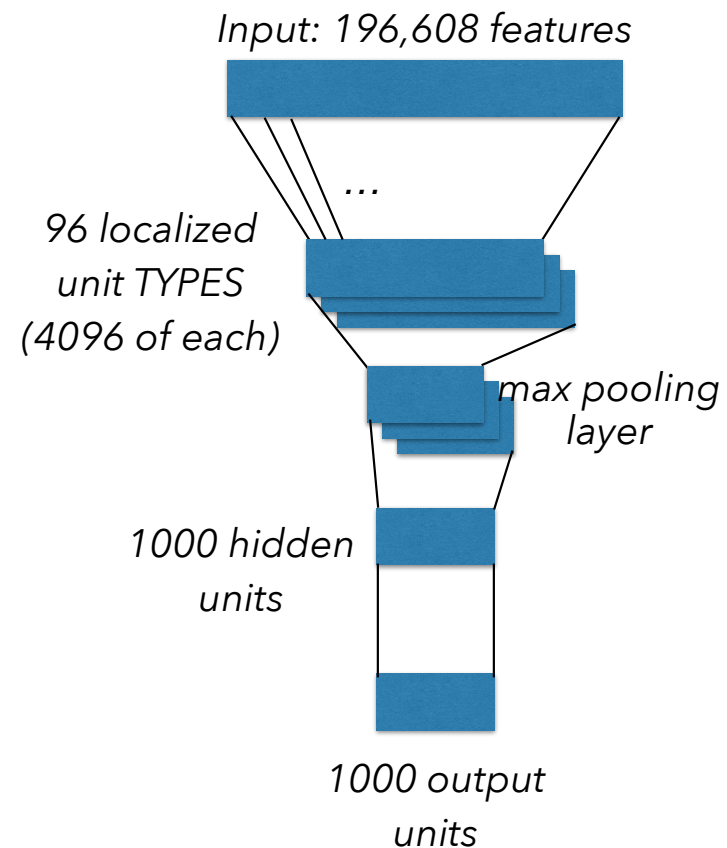
# Convolutional layer

- Convolutional layers enable us to detect whether the same feature appears at any location in the image
- But what we are ultimately concerned with is whether some feature (e.g. a dog) appears ***anywhere*** in the image



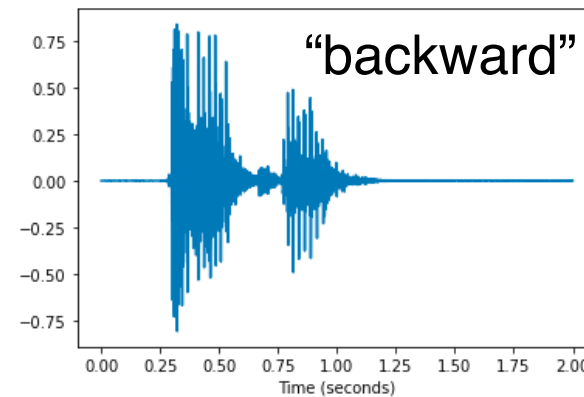
# Convolutional layer

- We can try to find things “anywhere in the image” with **max pooling**
- Instead of directly using the output of one convolutional layer as the input for the next, we reduce the “image size” at each layer by **pooling** the activations across adjacent units
- Taking the **maximum** across a group of units is like a logical OR, captures the “anywhere” property



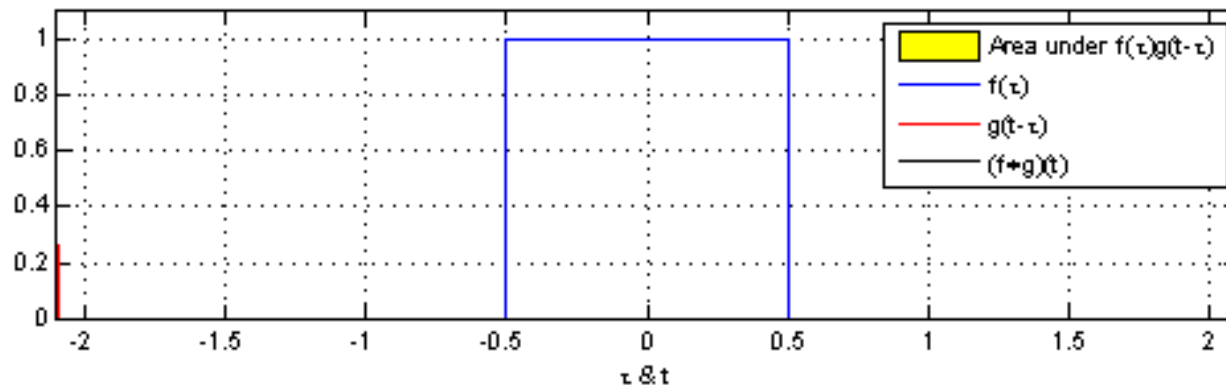
# Sound classification

- **Problem:** we want to classify *sounds*, represented as waveforms
  - \* Each sound comprises (up to) 16000 samples
  - \* Can we apply the same tricks as for images?



# Sound classification

- Solution: **1-D convolution!**



from <https://en.wikipedia.org/wiki/Convolution>

# Sound classification

- Solution: **1-D convolution!**

\* Assuming:  $f \in \mathcal{R}^T, \{h, g\} \in \mathcal{R}^n$  (f is length T vector, h and g are length n vectors)

\* Definition:  $(f \star g)[t] = \sum_{\tau=-\infty}^{\infty} f[t]g[t - \tau]$  (defining  $g[\cdot]=0$  for indices greater than n or less than 1)

\* Or:  $(f \star h)[t] = \sum_{\tau=1}^n f[t - \tau]h[\tau] = f[t - n : t] \cdot h$

# Sound classification

- We can stack multiple convolutional filters (aka kernels) into a **1-D convolutional layer**:
  - Let  $g_1 \dots g_k$  be a set of length- $n$  vectors, and  $x$  be a length- $T$  vector (input)
- Define

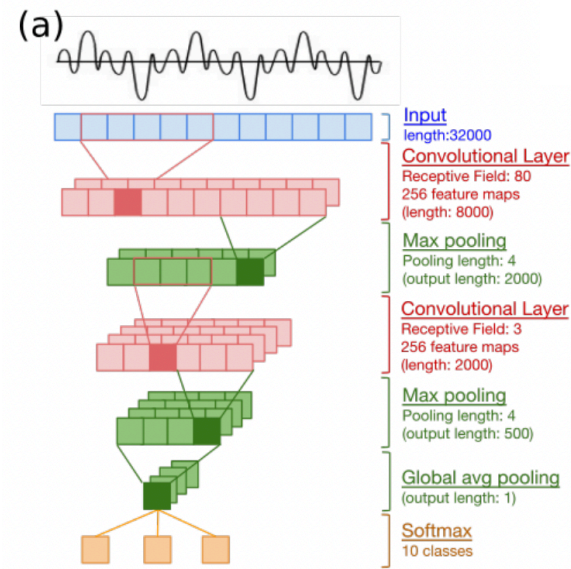
$$g(x) = \begin{bmatrix} g_1 \star x \\ g_2 \star x \\ \dots \\ g_k \star x \end{bmatrix}$$

# Sound classification

- Then we can:
  - apply a nonlinearity (e.g. ReLU) to the results
  - downsample the results using **max-pooling** or **average-pooling**
  - (maybe pass the results into a fully-connected layer?)

# Sound classification

- Example: **M3** (Dai et al., 2016)



# Sound classification

- 1-D convolutional networks!
  - (they're pretty much the same as 2-D convolutional networks, so far)
- You'll see a demo in lab!